

CAP Theorem

Chapter 5 | PostgreSQL Complete Guide Difficulty: Intermediate | Estimated Reading Time: 25 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: The CAP Theorem](#)
 - [The Three Properties Explained](#)
 - [Why You Can Only Choose Two](#)
 - [CAP in Real Systems](#)
 - [PACELC: The Extended Model](#)
 - [Consistency Models](#)
 - [ASCII Visual Diagrams](#)
 - [Practical Examples](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- State the CAP theorem and explain each of its three properties
 - Explain why a distributed system cannot simultaneously guarantee all three CAP properties
 - Classify well-known databases as CP, AP, or CA systems
 - Understand the PACELC model as a more nuanced extension of CAP
 - Apply CAP reasoning to architecture decisions in real-world distributed systems
-

Theory: The CAP Theorem

The **CAP Theorem**, proposed by computer scientist **Eric Brewer** in 2000 and formally proven by Gilbert and Lynch in 2002, states:

A distributed data store can only guarantee two of the following three properties simultaneously:

- *Consistency*
- *Availability*
- *Partition Tolerance*

This theorem is fundamental to distributed systems design and explains why different databases make different trade-off choices.

Why Does This Matter?

When designing systems that span multiple servers, data centers, or cloud regions, you will inevitably face network failures (partitions). The CAP theorem forces you to decide: when the network fails, do you want your system to **continue serving requests** (possibly with stale data) or **refuse requests** until consistency is restored?

The Three Properties Explained

C — Consistency

In the CAP theorem, consistency means **every read receives the most recent write or an error**.

- Every node in the distributed system reflects the same data at the same time
- If you write to node A, and immediately read from node B, you get the same value
- This is **linearizability** — the strongest consistency guarantee

Important: This is NOT the same as the C in ACID (where C means constraint/integrity consistency). CAP-consistency is about **replication consistency** across nodes.

A — Availability

Every request receives a non-error response — but the response might not contain the most recent write.

- The system always processes requests
- No node returns a timeout or error
- A response is always returned, even if it's slightly stale

P — Partition Tolerance

The system continues operating even when network messages are dropped or delayed between nodes.

- A "network partition" means some nodes cannot communicate with others
 - In a distributed system, partitions are inevitable (networks fail, packets are lost)
 - Any system spanning multiple machines **MUST** tolerate partitions
-

Why You Can Only Choose Two

In a distributed system, network partitions are not optional — they will happen. So **P is mandatory**. The real trade-off is between C and A during a partition:

During a network partition, you must choose:

Option 1 (CP): Stop serving requests until the partition heals
--> Consistent, but Unavailable during partition

Option 2 (AP): Keep serving requests with potentially stale data
--> Available, but possibly Inconsistent

The "CA" combination (sacrifice P) only makes sense for single-node systems where partitions are impossible — essentially, a traditional non-distributed RDBMS.

CAP in Real Systems

System	CAP Choice	Reason
PostgreSQL (single node)	CA	No distribution, no partition
PostgreSQL (with streaming replication)	CP	Synchronous replication halts on replica failure
MySQL (async replication)	AP	Reads from replica may be stale
Cassandra	AP	Tunable consistency, prefers availability
DynamoDB	AP (default) / CP (strong consistency option)	Configurable per-operation
HBase	CP	ZooKeeper coordinates, blocks during partition
Zookeeper	CP	Leader election, halts if quorum lost
MongoDB (replica set, majority reads)	CP	Majority reads guarantee consistency
CouchDB	AP	Master-less, eventual consistency
Etcd	CP	Raft consensus, blocks without quorum
Redis Cluster	AP	Async replication, may lose writes
Google Spanner	CP (effectively)	TrueTime API, globally consistent

PACELC: The Extended Model

CAP only addresses behavior during a partition. In practice, **latency vs. consistency** trade-offs exist even when no partition is occurring. The **PACELC model** (Daniel Abadi, 2012) extends CAP:

If Partition: A or C
Else: L (latency) or C (consistency)

System	PA/EC (During P: A; Normal: C)	PC/EL (During P: C; Normal: L)
DynamoDB	PA / EL	
Cassandra	PA / EL	
MongoDB		PC / EC
HBase		PC / EC
Megastore	PA / EC	

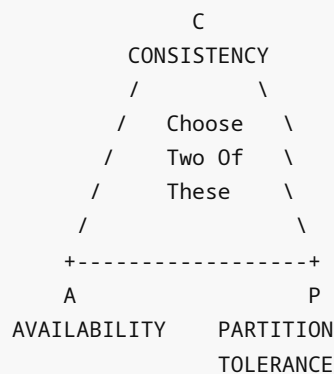
Consistency Models

From strongest to weakest:

1. **Linearizability (Strong Consistency):** Every operation appears instantaneous; reads always see latest write
2. **Sequential Consistency:** All nodes see operations in the same order, but not necessarily real-time
3. **Causal Consistency:** Causally related operations are seen in order; unrelated ops may differ
4. **Read-your-writes:** You always see your own writes
5. **Monotonic Read Consistency:** Once you've seen a value, you won't see an older value
6. **Eventual Consistency:** Given no new updates, all nodes will eventually converge to the same value

ASCII Visual Diagrams

Diagram 1: The CAP Triangle



CA Systems: Traditional RDBMS (PostgreSQL single node)
CP Systems: HBase, Zookeeper, Etcd, MongoDB (majority reads)
AP Systems: Cassandra, DynamoDB, CouchDB, Redis

Diagram 2: What Happens During a Network Partition

NORMAL OPERATION:

=====

Node A <----network----> Node B
[v=5] [v=5]

Client writes v=6 to A
A replicates to B --> B now has v=6
Both nodes consistent: [v=6] [v=6]

NETWORK PARTITION:

=====

Node A XXXX partition XXXX Node B
[v=6] [v=6 cached]

Client writes v=7 to A	
A cannot replicate to B (network cut)	
CP CHOICE (Consistency):	AP CHOICE (Availability):
=====	=====
A refuses the write	A accepts v=7
Returns error to client	B still serves old v=6
"System unavailable"	Clients get INCONSISTENT data
Waits for partition to heal	Eventually B will get v=7

Diagram 3: PostgreSQL Replication Consistency Options

```

ASYNC REPLICATION (AP-like):
=====
Primary --async--> Replica

Write to primary completes immediately
Replica may lag by seconds/minutes

If primary dies: replica promotes
--> Possible data loss (unflushed WAL)
--> Availability: HIGH
--> Consistency: EVENTUAL

SYNC REPLICATION (CP-like):
=====
Primary --sync--> Replica

Write confirmed only after replica ACKs
Zero lag (replica always current)

If replica dies: primary BLOCKS until timeout
--> Availability: LOWER
--> Consistency: STRONG (linearizable reads from primary)

synchronous_commit = on
synchronous_standby_names = 'replica1'

```

Practical Examples

Example 1: Banking System (Must be CP)

A bank transfer must be atomic and consistent:

- Debit account A AND credit account B
- During a network partition, the bank MUST block operations rather than allow inconsistent state
- **Choice: CP** — Availability is sacrificed to maintain consistency

Example 2: Social Media Feed (AP is acceptable)

A Twitter-like feed showing posts:

- If a user's post doesn't appear for 2 seconds on another device, it's acceptable
- The system should always serve the feed, even if slightly stale
- **Choice: AP** — Consistency is relaxed for high availability

Example 3: Shopping Cart (AP with reconciliation)

Amazon's Dynamo paper pioneered using AP for shopping carts:

- Multiple devices/sessions can add items independently
- On merge, all items are combined (no items are lost)
- This is eventual consistency with conflict resolution
- **Choice: AP** — Availability prioritized; conflicts resolved at checkout

PostgreSQL Commands

```
-- Check replication configuration
SHOW synchronous_commit;
SHOW synchronous_standby_names;

-- View replication status
SELECT
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    write_lag,
    flush_lag,
    replay_lag
FROM pg_stat_replication;

-- Check replication lag on replica
SELECT
    now() - pg_last_xact_replay_timestamp() AS replication_lag;

-- Configure synchronous replication (in postgresql.conf)
-- synchronous_commit = on    (CP: waits for replica ACK)
-- synchronous_commit = off   (AP-like: async, faster, may lose recent data)
```

SQL Examples

Example 1: Demonstrating Consistency with Transactions

```
-- ACID transaction ensures C in ACID (constraint consistency)
-- But in distributed setup, C in CAP = all nodes see same data

BEGIN;
```

```
-- Transfer $100 from account 1 to account 2
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
-- Either both happen or neither (atomicity)
COMMIT;
```

Example 2: Simulate Eventual Consistency Check

```
-- On primary: Check current value
SELECT balance FROM accounts WHERE account_id = 1;

-- On replica: Check if it matches (may lag in async setup)
-- \c postgresql://replica:5432/mydb
-- SELECT balance FROM accounts WHERE account_id = 1;
-- May return old value if lag exists
```

Example 3: Configure Synchronous Replication

```
-- On primary: Set synchronous replication
ALTER SYSTEM SET synchronous_commit = 'on';
ALTER SYSTEM SET synchronous_standby_names = 'replica1';
SELECT pg_reload_conf();

-- Verify
SHOW synchronous_commit;
```

Example 4: Check Replication Lag

```
-- On primary: see how far behind each replica is
SELECT
    application_name,
    client_addr,
    sync_state,
    ROUND(
        (sent_lsn - replay_lsn)::BIGINT / 1024.0 / 1024.0,
        2
    ) AS lag_mb,
    replay_lag
FROM pg_stat_replication
ORDER BY lag_mb DESC;
```

Example 5: Idempotent Operations (AP system strategy)

```
-- In AP systems, operations may be retried. Make them idempotent:

-- BAD: Not idempotent (running twice doubles the update)
UPDATE accounts SET balance = balance + 100 WHERE account_id = 1;
```

```

-- GOOD: Idempotent with idempotency key
CREATE TABLE payment_events (
    event_id    UUID PRIMARY KEY, -- Unique per payment attempt
    account_id  INT,
    amount      DECIMAL(10,2),
    processed_at TIMESTAMP DEFAULT NOW()
);

-- Only process if not already processed (idempotent)
INSERT INTO payment_events (event_id, account_id, amount)
VALUES ('uuid-here', 1, 100.00)
ON CONFLICT (event_id) DO NOTHING;

```

Example 6: Read-Your-Writes Consistency

```

-- Ensure you read from primary after a write (for read-your-writes consistency)
-- If using connection pooling to replicas, route writes to primary, then reads to
primary

-- Use session variable to track write timestamp
SET LOCAL synchronous_commit = 'on';

-- After write, check if replica has caught up before reading from it
SELECT pg_is_in_recovery(); -- FALSE on primary, TRUE on replica

```

Example 7: Two-Phase Commit (Distributed Consistency)

```

-- PostgreSQL supports 2PC for cross-database transactions
BEGIN;
-- ... operations ...
PREPARE TRANSACTION 'txn-001'; -- Phase 1: Prepare

-- If all participants prepared successfully:
COMMIT PREPARED 'txn-001'; -- Phase 2: Commit

-- Or rollback:
-- ROLLBACK PREPARED 'txn-001';

-- View pending prepared transactions
SELECT * FROM pg_prepared_xacts;

```

Example 8: Optimistic Locking (AP Systems Pattern)

```

-- Optimistic locking: detect conflicts at commit time
CREATE TABLE inventory (
    product_id INT PRIMARY KEY,
    qty         INT,

```

```

        version    INT DEFAULT 0
    );

-- Read (no lock)
SELECT qty, version FROM inventory WHERE product_id = 1;
-- Got: qty=100, version=5

-- Write (check version hasn't changed)
UPDATE inventory
SET qty = 95, version = 6
WHERE product_id = 1 AND version = 5;
-- If 0 rows updated, someone else changed it first --> retry

```

Example 9: Conflict-Free Replicated Data Types (CRDT Pattern)

```

-- G-Counter (grow-only counter) - conflict-free in AP systems
CREATE TABLE g_counter (
    node_id VARCHAR(50),
    counter BIGINT DEFAULT 0,
    PRIMARY KEY (node_id)
);

-- Each node only increments its own counter
UPDATE g_counter SET counter = counter + 1 WHERE node_id = 'node-1';

-- Total = sum of all node counters (always safe to merge)
SELECT SUM(counter) AS total FROM g_counter;

```

Example 10: Check Cluster Health

```

-- On a PostgreSQL cluster, check all replicas are healthy
SELECT
    CASE WHEN pg_is_in_recovery() THEN 'REPLICA' ELSE 'PRIMARY' END AS role,
    pg_postmaster_start_time() AS started_at,
    current_setting('server_version') AS version;

-- On primary: verify all expected replicas are connected
SELECT COUNT(*) AS replica_count FROM pg_stat_replication;

```

Common Mistakes

Mistake 1: Confusing CAP-Consistency with ACID-Consistency

CAP-C = all nodes have the same data (replication consistency) ACID-C = database constraints are never violated (integrity) These are completely different concepts. A system can be AP (not CAP-consistent) but still ACID-consistent within each node.

Mistake 2: Thinking You Can "Choose CA"

In a distributed system with multiple nodes, network partitions are inevitable. You cannot sacrifice P. The real choice is between C and A when a partition occurs.

Mistake 3: Applying CAP to Single-Node Systems

CAP only applies to distributed systems. A single PostgreSQL instance is neither CP nor AP — it's just consistent (within its ACID guarantees) and has no partition concern.

Mistake 4: Treating Eventual Consistency as "Broken"

Eventual consistency is a valid and useful guarantee. Many systems (DNS, social media feeds, CDNs) use eventual consistency successfully. The key is understanding what your application can tolerate.

Mistake 5: Ignoring the PACELC Trade-offs

Engineers focus on CAP (partition behavior) but ignore latency vs. consistency trade-offs during normal operation. Synchronous replication is "more consistent" but adds 5-50ms to every write.

Mistake 6: Not Planning for Partition Recovery

Systems must plan for what happens when a partition heals. How are conflicts resolved? Which node's data wins? Does the system use last-write-wins, vector clocks, or application-level merge logic?

Best Practices

1. **Understand your consistency requirements first:** Not every system needs strong consistency. Identify which operations need it and which can tolerate eventual consistency.
2. **Use synchronous replication for critical data in PostgreSQL:** For financial data, use `synchronous_commit = on` with at least one synchronous replica.
3. **Design idempotent operations:** In AP systems, operations may be executed multiple times during partition recovery. Make them idempotent.
4. **Add idempotency keys:** Use unique event IDs to prevent duplicate processing.
5. **Monitor replication lag:** In async setups, alert when replica lag exceeds your SLA threshold.
6. **Test partition scenarios:** Use chaos engineering (kill a node, cut network) to verify your system behaves as expected during partitions.
7. **Document your consistency guarantees:** Tell your API consumers what consistency level they should expect.

Performance Considerations

- **Synchronous replication** adds write latency equal to the network round-trip to the replica (typically 1-50ms). This is the price of CP.
- **Asynchronous replication** has near-zero write overhead but risks data loss if the primary crashes before replication.
- **Read from replicas** scales read throughput horizontally — but only safe if your application tolerates stale reads.

- **Quorum reads/writes** (used in Cassandra) balance latency and consistency: write to W nodes, read from R nodes; consistent if $W + R > N$.

Quorum example ($N=3$ replicas):

$W=2, R=2 \rightarrow W+R=4 > 3 \rightarrow$ Consistent reads

$W=1, R=1 \rightarrow W+R=2 = 2 \rightarrow$ May read stale data

$W=3, R=1 \rightarrow W+R=4 > 3 \rightarrow$ Consistent reads but slow writes

Interview Questions

1. What does CAP stand for and what does it mean?
2. Why can't a distributed system have all three CAP properties?
3. What is the difference between CAP-Consistency and ACID-Consistency?
4. Classify Cassandra and HBase according to CAP.
5. What is PACELC and how does it improve on CAP?
6. What does "eventual consistency" mean?
7. How does PostgreSQL fit into the CAP theorem?
8. What is a network partition?
9. What is linearizability?
10. How would you design a shopping cart that works during network partitions?

Interview Answers

Q1: CAP = Consistency (every read returns the latest write), Availability (every request gets a response), Partition Tolerance (system works despite network failures). In a distributed system, you can guarantee at most two simultaneously.

Q2: Proof sketch: During a network partition, nodes A and B cannot communicate. If a write comes to A and a read comes to B simultaneously: if you return a response from B (availability), it may return old data (inconsistency). If you want consistency, B must refuse the request (unavailability). You cannot return a consistent response from B without access to A.

Q5: PACELC

PACELC extends CAP: during a Partition (P), choose Availability (A) or Consistency (C). Else (E, normal operation), choose Latency (L) or Consistency (C). For example, Cassandra is PA/EL: favors availability during partitions, and low latency during normal operation (asynchronous replication). Google Spanner is PC/EC: consistent during partitions and during normal operation (at the cost of latency).

Q10: Shopping Cart Design

Use AP with merge semantics. Each device has its own cart replica. Items are added to a set (merge = union). On checkout, merge all replicas by taking the union of all items. This is a CRDT (Conflict-Free Replicated Data Type). No item is ever lost, and the system is always available. The only "conflict" (same item added twice) is resolved by deduplication.

Hands-on Exercises

Exercise 1: Replication Setup

Configure PostgreSQL streaming replication. Set `synchronous_commit = on`. Observe that writes to the primary wait for replica acknowledgment.

Exercise 2: Simulate a Partition

In a replication setup, pause the replica (simulate partition). Observe:

- With async replication: primary keeps accepting writes (AP)
- With sync replication: primary blocks after timeout (CP)

Exercise 3: Replication Lag Query

Write a monitoring query that alerts when replication lag exceeds 30 seconds.

Exercise 4: Idempotent Insert

Write an idempotent UPSERT that processes payment events. Run it twice with the same `event_id` and verify only one row is created.

Exercise 5: Optimistic Locking

Implement optimistic locking for an inventory table. Simulate two concurrent users trying to reserve the last item.

Solutions

Solution 3

```
-- Alert if any replica is more than 30 seconds behind
SELECT
    application_name,
    client_addr,
    replay_lag,
    CASE
        WHEN replay_lag > INTERVAL '30 seconds' THEN 'CRITICAL: Lag too high'
        WHEN replay_lag > INTERVAL '10 seconds' THEN 'WARNING: Elevated lag'
        ELSE 'OK'
    END AS status
FROM pg_stat_replication;
```

Solution 4

```
CREATE TABLE payment_events (
    event_id    UUID PRIMARY KEY,
    account_id  INT NOT NULL,
    amount      DECIMAL(12,2) NOT NULL,
    processed_at TIMESTAMP DEFAULT NOW()
);

-- Idempotent: second execution does nothing
INSERT INTO payment_events (event_id, account_id, amount)
```

```
VALUES ('a1b2c3d4-...', 101, 250.00)
ON CONFLICT (event_id) DO NOTHING;

-- Verify only one row
SELECT COUNT(*) FROM payment_events WHERE event_id = 'a1b2c3d4-...';
```

Solution 5

```
-- Session 1
BEGIN;
SELECT qty, version FROM inventory WHERE product_id = 1 FOR UPDATE;
-- qty=1, version=10

-- Session 2 (concurrent)
BEGIN;
SELECT qty, version FROM inventory WHERE product_id = 1;
-- Also sees qty=1, version=10

-- Session 1 commits first
UPDATE inventory SET qty = 0, version = 11 WHERE product_id = 1 AND version = 10;
COMMIT; -- Succeeds, 1 row updated

-- Session 2 tries to commit
UPDATE inventory SET qty = 0, version = 11 WHERE product_id = 1 AND version = 10;
-- 0 rows updated! Version mismatch. Must retry.
```

Advanced Notes

Consistency vs. Coherence

Consistency (CAP): All nodes agree on the same value for a data item. **Coherence**: A weaker guarantee — each individual data item is consistent, but the database as a whole may not be transactionally consistent.

Jepsen Testing

Kyle Kingsbury's Jepsen project tests distributed databases for CAP violations. Many systems that claim CP have been shown to lose data or return stale reads under network partition conditions. Always verify claims empirically.

PostgreSQL and the "CA" Myth

PostgreSQL documentation sometimes describes it as "CA" but this only applies when run as a single node. With Patroni or other HA solutions, PostgreSQL streaming replication with synchronous commits makes it a CP system. With async replication and hot standby reads, it behaves more like AP.

Cross-References

- **Previous:** [04 - OLTP vs OLAP](#)
- **Next:** [06 - ACID Properties](#)
- **Related:** [07 - BASE Properties](#)

- **Related:** [02 - Types of Databases](#) — CAP classifications of different DB types
- **See Also:** [09 Administration/04 replication.md](#) — Setting up PostgreSQL replication
- **See Also:** [06 - ACID Properties](#) — How ACID relates to CAP-Consistency